

MENTOR-RL Methods

Peter Kruse

June 2026

0.1 Introduction

Mechanistic interpretation identifies relationships among molecular entities such as genes, proteins, and RNA. Systems biology studies their interactions to find *modules*, or groups of functionally related entities, and characterize the mechanisms underlying their behavior [17, 5]. Because these interactions span scales and regulatory layers, modern analysis uses multiplex networks with the same genes connected by different edge types [4, 28, 14] and propagation methods such as random walk with restart (RWR) to identify functional neighborhoods [22, 13]. Such analyses support disease-mechanism discovery, drug-target prioritization, and polygenic-phenotype interpretation [27]. Methods including iterative random forest [6, 12] and MENTOR [44] construct biologically informed networks and modules, but the resulting multiscale graphs can still be too large to explore systematically. Their combinatorial complexity grows while human interpretive capacity remains fixed.

Large language models (LLMs) can synthesize this information, but frontier inference cost grows with exploration length and reasoning depth. Open-weight models offer a less costly alternative: intermediate reasoning improves reliability [49, 23], external tools provide deterministic verification and provenance [39, 51], and explicit self-verification further improves factuality [26, 42, 25]. These ideas have produced gene-function embeddings [10], database-backed self-verifying pipelines [48], and multi-agent systems for mechanistic synthesis [47, 25]. However, these systems still depend on repeated frontier-model inference and are not trained end-to-end as policies over verifiable biological rewards.

Reinforcement learning supplies the machinery to close that gap. Direct preference optimization (DPO) aligns a policy with pairwise judgments [11, 33, 36], while policy-gradient methods such as proximal policy optimization and group relative policy optimization (GRPO) optimize scalar rewards over complete rollouts [40, 41]. Verifiable rewards can elicit long-horizon reasoning [16], and related methods train tool-using agents for browsing, APIs, and multistep web tasks [30, 35, 34]. They have not yet been applied to a policy grounded in network-derived biological targets.

Current systems also treat networks as static objects to query rather than learning the rules they obey, so little carries over to a new cell type, tissue, or disease. Sequence models trained on synthetic tasks can instead form internal, causal models of a generating process and generalize to unseen states [24]. The compact, recurring rules can live in model weights; irreducible content cannot. Thus, the model should learn operators that regenerate structure while biological edges, walk scores, and distances remain in a deterministic runtime. Learning such a world model also requires dense sampling of the generation process rather than fixed templates over one network.

We therefore propose MENTOR-RL, an open-weight model trained on a structured lattice of multiplex networks. In every context, selected predictive expression network (PEN) layers define the gene universe, the remaining layers are induced on that universe, and all layers are stacked. Varying the cell type, tissue, or brain region changes the network while holding this construction rule fixed. This *multiplex lattice*

supplies the dense sampling that a single network cannot. The invariant is the operator that maps a multiplex to its structure and the way layer composition drives it. Hierarchies should differ across contexts; this divergence is biological signal, and recovering it tests whether the model learned the operator rather than one conserved hierarchy.

A hierarchy alone cannot serve as the world model, because it is a lossy abstraction of the multiplex it summarizes. Two clades that sit far apart in a tree may still be joined by short, biologically supported paths through bridge nodes in the underlying network, which is why a signal such as drug-target enrichment can appear in several distant branches and yet act through a single connected system. The model must hold the network itself and reason about the cross-clade connectivity that the tree suppresses. We use the hierarchy to find and prioritize candidates, and the network to confirm them.

This proposal pursues four objectives: define an ontology-grounded multiplex lattice; train a world model on three co-equal structural views using typed, runtime-validated targets; post-train that checkpoint as a tool-using agent with deterministic rewards; and test the world-model claim through two flagships—hierarchy generation for an unseen context, and projection of a target’s RWR-LOE neighborhood onto the hierarchy followed by network-connectivity recovery. Activation probes test whether the recovered structure is causal. Together these objectives aim to make mechanistic discovery broader, faster, and more context-aware.

0.2 Proposed Methods

0.2.1 Multiplex Lattice

In this section, we define the multiplex lattice that serves as the substrate for our mechanistic world model. The lattice comprises multiplex networks in which entities such as genes and proteins are nodes and their context-specific interactions are edges. We select PEN layers for a context to define its *gene universe*, the set of genes over which the network operates. The remaining non-PEN layers are then *induced* on that universe, restricting them to genes present in the PEN layers and the edges among them. Finally, all layers in the context are *stacked* to form a multiplex network that captures multilayer interactions among genes. Thus, there is no single shared backbone across the lattice; even layers drawn from a common source become context-specific once induced. Two sampled points in the lattice may contain different nodes, edges, and therefore hierarchies. For the scope of this proposal, we build the lattice on the brain, for which we hold a pre-existing whole-brain multiplex assembled by the same rule. The brain multiplex combines PEN layers over the brain gene universe with a HumanNet V3 backbone [21] induced on it.

The lattice comprises a tiered structure that captures the relationships between different contexts. We visualize an example brain multiplex lattice in Figure 1. Its

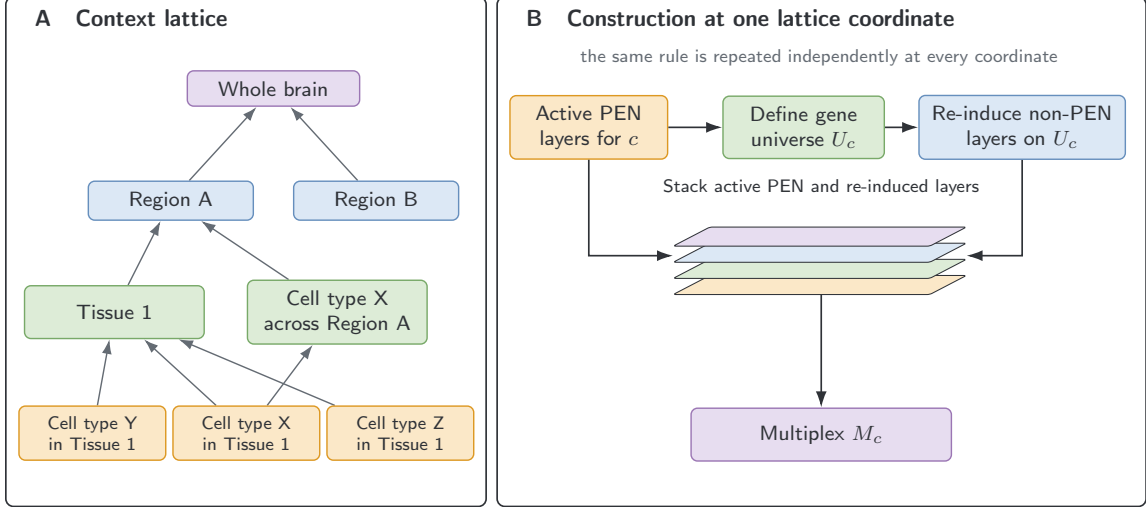


Figure 1: The multiplex lattice and its shared construction rule. Panel A gives a generic partial-order example: a cell-type-within-tissue context belongs to both its tissue context and its cell-type-across-region context. Panel B shows the construction at one coordinate. Active PEN layers define the gene universe, non-PEN layers are re-induced on that universe, and both layer sets are stacked into a distinct multiplex.

most granular component is a single cell type, such as an excitatory neuron, inhibitory neuron, or glial cell, within one tissue of one region. Combining cell types yields a tissue-level multiplex, combining tissues yields a region-level multiplex such as the prefrontal cortex, hippocampus, or cerebellum, and combining regions builds up to the whole-brain multiplex described above. Every tier is therefore a multiplex in its own right, built by the same rule, rather than a label over the tier below. These tiers do not form a strict tree, since a context can belong to more than one grouping at the tier above it: an astrocyte in CA1 sits beneath both the all-cell-types node for CA1 and the astrocyte node for the hippocampus as a whole. Section 0.2.2 formalizes the resulting structure. Disease networks for anxiety, suicide, and pan-substance-use-disorder are points in this same lattice, which makes transfer meaningful: a model that has learned the construction operator can be applied to them directly.

Higher tiers are built by applying the same rule to a union of contexts. We take the union of the parts’ gene universes, re-induce the non-PEN layers on it, and stack the parts’ PEN layers on top. This process makes lattice sampling highly combinatorial, as changing a single PEN layer changes two things at once. The gene universe grows or shrinks, and the backbone is re-induced to match. A pair of multiplexes that differ by one layer therefore also differ by the backbone that layer brought with it. Measuring the effect of a layer on its own requires holding the gene universe fixed, which we do via controlled ablation. The lattice also holds far more combinations than we can build, so tier counts and sampling rates stay provisional until divergence validation shows how much contexts actually differ.

0.2.2 Context Ontology

To train MENTOR-RL reproducibly, we define a *context ontology* that specifies relationships among contexts. This ontology is supplied, versioned metadata that lets us name, version, and align the lattice with external sources. We use several grounded ontologies to define the context hierarchy. Anatomy is defined by UBERON [29], while cell types are defined by the Cell Ontology [15], extended by the Provisional Cell Ontology [45] for transcriptomic brain types not yet represented in the Cell Ontology. Developmental stages are defined by HsapDv [7], anatomy-to-cell-type mappings by the Human Reference Atlas [9], and node metadata by the CELLxGENE schema [1]. Because the Human Reference Atlas covers the whole body, this paradigm can later extend beyond the brain.

The context ontology defines a partial order, not a tree, over contexts. Because some cell types are present in multiple tissues and some tissues are present in multiple regions, a strict hierarchy is not possible. Instead, we define a *join* operation over the ontology: the join of two contexts is the smallest context containing both. This operation ties the ontology to the construction rule. Joining two contexts names the union multiplex described in Section 0.2.1: the ontology gives the address, and the construction rule builds the network.

The model reads the ontology as supplied input; the ontology is never stored in the weights. Instead, the model learns two things: to navigate the ontology by returning a context’s parents, children, siblings, joins, and distances, and to apply the construction rule that turns a context into a multiplex. The ontology tells the model where it is, and the construction rule tells it what to build there. Because only the terms change, the same navigation transfers when the lattice extends to other organ systems.

0.2.3 Three Structural Views

To represent this ontology-based lattice structure as a training target, we define three equally weighted structural views of a multiplex network: bottom-up random-walk-with-restart lines-of-evidence (RWR-LOE) neighborhoods, top-down MENTOR-EV-derived hierarchies, and network connectivity. Each view represents the same underlying multiplex with different biases.

The RWR-LOE view is a bottom-up representation of a multiplex that captures the local topology around each node [20]. For each node in a given multiplex, we perform a random walk with restart over all layers of the multiplex, treating each layer as an independent line of evidence. The walk returns a probability distribution over all nodes in the network, with higher probabilities representing closer proximity to the seed. We then threshold the distribution using a geometric elbow, which cuts the score-versus-rank curve at its bend. The genes above the cutoff form a module of genes that are proximal to the seed over multiple lines of evidence. From RWR-LOE

modules, we can learn several operator types, including rank and score lookup, rank comparison, and relations between modules.

MENTOR-EV-derived modules represent an alternative, top-down view of the multiplex. MENTOR-EV applies MENTOR [44] using every gene in the multiplex as a seed, so the resulting hierarchy spans the full gene universe rather than a supplied gene set. To create MENTOR-EV modules, we first use RWR to compute the probability distribution for each node in the multiplex. We then compute pairwise similarity between RWR vectors using Spearman correlation and subtract those correlations from one to construct a dissimilarity matrix over all nodes. Finally, hierarchical clustering produces a dendrogram in which leaves are genes and each internal node defines a *clade*, the set of genes beneath it. Cutting the dendrogram at a given merge height produces a family of nested modules and controls their granularity. From MENTOR-EV modules, we can learn several operator types, including module membership, module comparison, lowest common ancestors, and distance between modules.

Every one of these targets is read from a single dendrogram. The structural targets are alignment-free and do not require two hierarchies to agree, although we compare derived divergence summaries across contexts. This is what makes the divergence described in Section 0.2.1 trainable rather than an obstacle: hierarchies differ across contexts, but each one supplies its own targets.

However, RWR-LOE and MENTOR-EV modules are both lossy abstractions of the underlying multiplex structure. Two clades on distant branches of the dendrogram may still be connected by short paths through a small number of bridge nodes, and a neighborhood cut at the elbow drops genes that remain reachable in a few hops. As such, we also include learnable network connectivity operators as a third view, which include cross-clade path recovery, connector and bridge node identification, path decomposition by supporting layer, and cohesion tested against a null. These operators allow the model to reason about the underlying multiplex structure and recover connectivity that is not captured by the other two views.

0.2.4 Stage 1: Multiplex World Model

In Stage 1, we train a world model to internalize the multiplex lattice by learning the operators that generate its structure.

Deterministic Runtime and Tool Contract

To interface the lattice environment with the training pipeline, we use a compiled native multiplex library exposed to Python through `ctypes` bindings [19]. Corpus construction, training, and optimization code are implemented in Python [38], while graph storage and graph operations are handled by the native runtime for efficient execution on large multiplexes. Multiplex networks are persisted as raw, CSR-style binary stores described by JSON manifests; this binary format is confined to

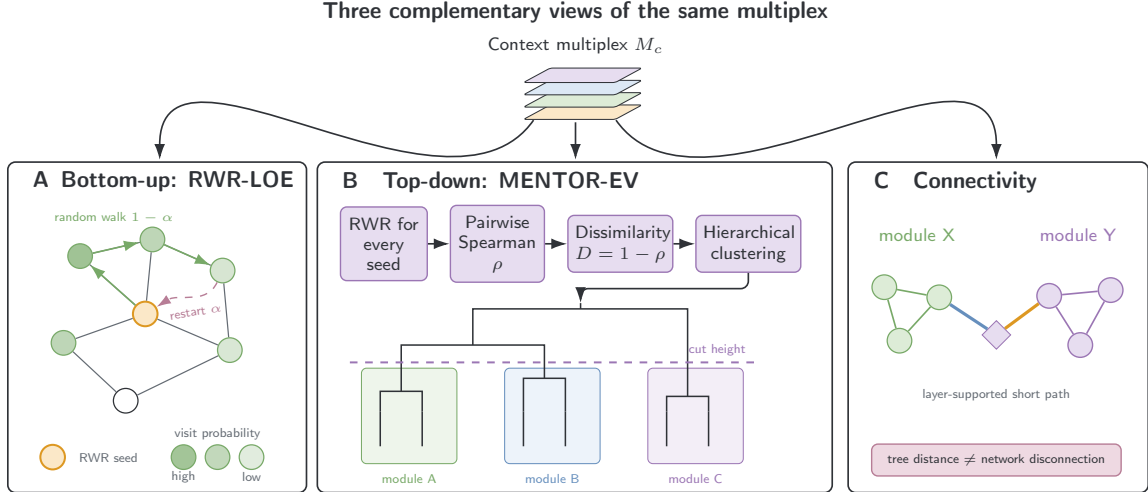


Figure 2: Three structural views derived from the same context multiplex. RWR-LOE supplies bottom-up neighborhoods, MENTOR-EV supplies top-down nested clades, and direct network operators recover paths and bridge nodes suppressed by either abstraction. The views are complementary targets rather than interchangeable descriptions.

native storage and execution. Model-facing tools instead accept structured biological requests and return compact, structured observations with provenance. The manifest carries the layer names, node metadata, and build metadata needed to load the graph.

The runtime uses a set of optimized multiplex functions developed as part of the MENTOR [44] and MENTOR-IA [47] projects, known as RWR++. Core operators include neighborhood retrieval, shortest-path search, random walk with restart on both monoplex and multiplex views, and subgraph induction. These operators provide the primary graph-facing actions available to the model.

We structure graph tools as learnable, interpretable operators according to three rules. First, each tool accepts only biologically meaningful arguments, such as gene identifiers, layer names, and module identifiers. Thus, when learning to produce a tool call, the model reasons about the biology rather than the file system or command-line interface. Second, we defer numeric computation to the runtime. Because tokenization makes precise floating-point values difficult for the model to produce, the model calls a tool to compute any required statistic. Third, we enforce determinism: the same arguments, graph version, layer scope, and parameter version always return identical output. This enables tool-call caching and reproducible scoring.

Runtime operators expose compact graph summaries and provenance metadata to the model rather than raw vectors or matrices. These observations record the relevant layer identity, operator parameters, and source information needed to trace the final interpretation back to the graph evidence used to construct it. To ensure consistency across serialized multiplex files, C++ runtime objects, Python wrappers, and logged

training artifacts, all nodes are represented in a single canonical identifier space and all edges are referenced using deterministic, layer-aware identifiers. Table 1 lists the model-facing tool vocabulary used in the curriculum.

Table 1: Model-facing RWR++ tool vocabulary. Related calls are grouped by function while preserving the tool names the model emits. The final column gives the curriculum step at which each group is introduced (Section 0.2.4).

Tools	Purpose	Primary inputs	Step
<code>get_neighbors;</code> <code>induce_subgraph</code>	Direct neighbors of a gene and the edges induced within a gene set	gene or gene set; layer scope	S2
<code>get_gene_layers;</code> <code>get_nodes_by_layer;</code> <code>get_layer_stats;</code> <code>get_component_summary</code>	Layer membership, presence of a set in a layer, per-layer statistics, and connected components	genes; layer identifiers	S2
<code>shortest_paths;</code> <code>get_path_layer_counts</code>	Shortest paths within and across layers, and the per-layer edge counts supporting a path	source and target genes; path	S3
<code>rwr;</code> <code>rwr_loe</code>	Random-walk expansion from seeds, and leave-one-out relatedness with its elbow cutoff	seed genes; query genes; top_k	S4
<code>get_rank;</code> <code>get_distance;</code> <code>get_spearman;</code> <code>get_pearson;</code> <code>get_dot_similarity</code>	Rank of a target gene from a seed, and pairwise distance or similarity under a stated metric	source and target genes; metric	S4
<code>get_rank_vector_summary;</code> <code>get_encoding_summary</code>	Compact summaries of rank-vector and encoding outputs without exposing dense vectors	seed or query genes; layer scope	S4
<code>get_seed_essentiality;</code> <code>get_layer_ablation;</code> <code>get_node_perturbation</code>	Effect of removing a seed, a layer, or a node on the resulting neighborhood	seed genes; layers; genes	S4, S6
<code>get_clade;</code> <code>get_lca;</code> <code>get_subtree;</code> <code>get_clade_relations</code>	Clade membership, lowest common ancestor of a gene pair, the subtree induced by a cut, and parent, child, and sibling relations	clade or module identifier; gene pair; cut height	S5
<code>rwr_netstats</code>	Module and cohesion statistics, with an empirical p -value where applicable	module identifiers; statistic; null size	S5, S6
<code>query_mygene;</code> <code>enrich_gene_set</code>	Gene annotation, and functional or module enrichment of a gene set	query or gene set; against; thresholds	S7

Supervised Curriculum

To train the model to internalize the multiplex lattice, we design a supervised curriculum that progressively teaches it to navigate the lattice and learn the operators that generate its structure. Two rules govern the curriculum. First, we use closed-book training only for bounded, stable conventions. Identifiers such as layer tags, layer families, module-identifier formats, and direction conventions are small, finite, and stable, so memorizing them through training is acceptable. Second, open-book training means reading, not computing. We read unbounded and unstable outputs, such as continuous values and statistics, from the runtime rather than memorizing them. Later, this rule extends to tool-call training, in which the model learns to parse returned objects instead of restating them from memory. This distinction supports generalization: the model should learn the operators that generate structure, not the specific values they produce.

Using these rules, we define a ten-step supervised training curriculum that gradually introduces more complex operators and tasks. Each stage builds on the previous one, starting with basic graph navigation and moving toward module comparison, path recovery, and enrichment analysis. We group similar stages in the prose; Table 2 presents the full curriculum.

Curriculum stages S0–S3 focus on foundational graph-navigation operators. S0 covers Ensembl-to-symbol and symbol-to-Ensembl conversions, which are necessary for all downstream tasks. S1 introduces the lattice coordinate system, teaching the model to navigate the multiplex structure, retrieve layer information, and interpret rankings and metrics. S2 covers atomic graph facts and negative examples, such as edge existence, neighbors, layer membership, and degree. S3 introduces path-based and cross-layer operators, including connector and bridge nodes.

After foundational training, stages S4–S6 focus on the three structural views of the multiplex. S4 teaches the model to operate on bottom-up RWR-LOE neighborhoods, S5 trains it on top-down hierarchical modules, and S6 unifies these views through network connectivity. S5 also includes examples that teach correspondences between the RWR-LOE and MENTOR-EV views, preventing them from being learned as unrelated skills.

Stages S7 and S8 build learned lattice structure concepts by adding projection and enrichment tasks. S7 composes RWR-LOE and MENTOR-EV operators into the projection workflow. First, a target gene seeds RWR-LOE to produce its network neighborhood. The runtime then enriches that neighborhood against every level of the MENTOR-EV hierarchy, yielding an enrichment score and significance value for each clade. High enrichment across several distant clades is evidence of a single connected system rather than unrelated biological processes. This reinforces the idea that tree distance does not mean that clades are unrelated (Section 0.2.3). Therefore, recovering bridge nodes, short cross-clade paths, and their supporting layers is essential. S8 teaches composition and context-ontology navigation in open-book mode. Given a supplied partial order, the model returns a context’s parents,

children, and siblings, computes joins and ontology distances, and uses each join to name the corresponding union multiplex. It also learns to predict how neighborhoods, clades, and connectivity change when a layer is added or removed, which is crucial for understanding differences across sampled lattice points.

Finally, S9 consolidates the curriculum by combining earlier stages and preparing the checkpoint for the multistep reasoning required in Stage 2. The model trains on tool-call construction, selection, parsing, provenance, and task refusal, all of which are necessary for the mechanistic agent (Section 0.2.5) to operate effectively in the multiplex lattice.

The curriculum scales difficulty along several axes. We scale the lattice tier from single cell types to tissues, regions, and eventually the entire brain. Cut-height granularity controls module size, progressing from coarse to fine-grained views of clades. We also scale operator complexity and book mode explicitly. To measure progress, we define task-specific gates that the model must pass before advancing to the next stage. All gates use held-out question phrasings and graph regions for each family, as well as negative examples that measure abstention.

Corpus Generation and Validation

To build the training corpora for each curriculum stage, we generate questions procedurally from various points on the lattice. By sampling many contexts and layers, we allow the model to see each operator applied to many different networks, which encourages generalization. All targets have their ground truth produced by the deterministic runtime defined in Section 0.2.4, and examples are cached with their layer descriptor, metric, seed set, and graph version to ensure reproducibility. We freeze context, clade, and tuple assignments before generating training examples, then materialize each curriculum corpus on demand. Performance at one stage may change the sampling strategy or corpus volume within the frozen training partition, but it cannot change which units belong to training, validation, or test.

We validate every answer field against a typed validator, and each question family composes the validators its fields require. A path field, for example, is accepted only when the node sequence is a valid walk whose length matches the runtime shortest path, so any optimal path is accepted rather than a single reference answer. A claim field is accepted only when the stated claim appears in the enumerated allowed set and no disallowed claim is asserted; this mechanism produces the calibration negatives in S2. Validation is all-or-nothing: an example enters the corpus only if every required field passes. The same validators score evaluation outputs, providing deterministic ground truth and consistent measurement across contexts and curriculum stages.

To provide sufficient training signal, our provisional lattice corpus includes 24 contexts and a full-brain multiplex. We expect to generate approximately 3 million supervised examples: roughly 10% closed-book, 75% open-book, and 15% tool-call examples, totaling about 1.2 billion training tokens. Cached ground truth, rather than text, dominates storage, with an estimated 2–4 terabytes across the 24 contexts.

Table 2: The ten-stage supervised curriculum. Book mode indicates whether targets are memorized (closed), read from supplied context (open), or produced by constructing and parsing a tool call. Data shares are provisional and will be reset once divergence validation establishes how strongly contexts differ across the lattice.

	Focus	Book mode	Share	Gate
S0	Entity normalization between symbols and Ensembl identifiers, including ambiguous symbols	closed	4%	Near-perfect accuracy on held-out identifiers, and correct deferral on ambiguous symbols
S1	Lattice coordinate system: layer tags and families, module identifiers, and direction conventions	closed	4%	Near-perfect accuracy on held-out tags and module identifiers
S2	Atomic graph facts: edge existence, neighbors, layer membership, degree and hubness, components, and calibration negatives	open	14%	Per-family accuracy on held-out graph regions and phrasings, and correct refusal on calibration negatives
S3	Paths and cross-layer routes, path decomposition by layer, and connector and bridge nodes	open	8%	Path and bridge recovery on held-out graph regions
S4	Bottom-up RWR-LOE neighborhoods: rank and score lookup, comparison, the elbow, leave-one-out support, and stability	open, tool	16%	Recovery and comparison accuracy stratified by distractor band
S5	Top-down MENTOR-EV hierarchy: clade membership, lowest common ancestor, subtree at a cut, and correspondence with RWR-LOE	open	16%	Hierarchy-navigation accuracy across cut heights, and correct cross-source containment
S6	Cross-clade connectivity: short paths between distant clades, bridge nodes, layer support, and cohesion against a null	open	12%	Cross-clade path recovery, and correct rejection of the false-separation reading
S7	The projection operation: seed RWR-LOE from a target gene, enrich its neighborhood against every hierarchy level, and return to the network	open, tool	12%	Correct projection, connectivity recovery, and interpretation, scored separately
S8	Composition and lattice scaling: open-book context-ontology navigation, joins, distances, and structural changes when layers are added or removed	open, tool	8%	Parent, child, and sibling set accuracy; join and ontology-distance accuracy; and correct structure on unions assembled from parts seen in training
S9	Tool-call consolidation: selection, argument construction, result parsing, provenance, and refusal of raw path arguments	tool	6%	Schema-valid tool-call rate and correct parsing on held-out tools and phrasings

This provides a second reason to materialize corpora in curriculum order rather than all at once. These estimates are provisional and will be adjusted based on results at each curriculum stage.

Splits and Leakage Control

To evaluate generalization, we control leakage among the training, validation, and test splits. Although each context contains a separate network, these networks share genes

and therefore some underlying edges. We define three levels of holdout validation: context, clade, and tuple. To hold out a context, we partition lattice points into training, validation, and test sets without splitting a context internally by gene. We hold out two to four contexts entirely from the 24-context lattice so that their hierarchies are never seen during training.

In addition to contexts, we hold out clades by selecting maximal subtrees at a chosen anchor height. This is necessary because a gene appears in every clade along its hierarchical ancestry; therefore, splitting by individual clades could leak membership information due to the dendrogram’s nested structure.

Finally, we partition specific query units into separate splits. A query unit is a seed with its target set, together with the cut height for hierarchical modules. We validate these test tuples by checking their Jaccard overlap with training tuples. A test tuple is permissible only if its target set has a Jaccard overlap below a provisional 0.2 with any training tuple sharing the same seed. Two further controls apply at this level. We cap how many high-degree seeds enter any single split, because RWR-LOE neighborhoods around hub genes overlap heavily even when contexts and clades are cleanly separated. We also keep both members of an S8 minimal pair on the same side of the split, so that a held-out multiplex is never trivially inferable from a partner that differs from it by a single PEN layer.

To enforce these controls, we audit the training, validation, and test splits before training. We check gene overlap between train and test sets, the fraction of test clades whose ancestry appears in training, the representation of hub nodes, and the number of minimal pairs split across sets, which should be zero. If any check fails, we correct the assignments and regenerate the affected corpora before freezing the split. One source of leakage remains: because contexts share genes, they also share some backbone edges. We bound and report this component rather than remove it because the invariant across the lattice is the construction operator, not any particular backbone. This process ensures that the model is evaluated on unseen data and that any remaining leakage is quantified and recorded before evaluation.

0.2.5 Stage 2: Mechanistic Agent

After training MENTOR-RL on all curriculum stages, we use the resulting checkpoint to initialize a mechanistic agent that explores the multiplex lattice and generates mechanistic interpretations. The agent is post-trained through preference optimization and reinforcement learning on long-horizon reasoning tasks. We formalize the agent loop and structured state in Section 0.2.5 and describe the task cases, corpora, and reward design in Sections 0.2.5–0.2.5. Section 0.2.6 details the full training pipeline and curriculum.

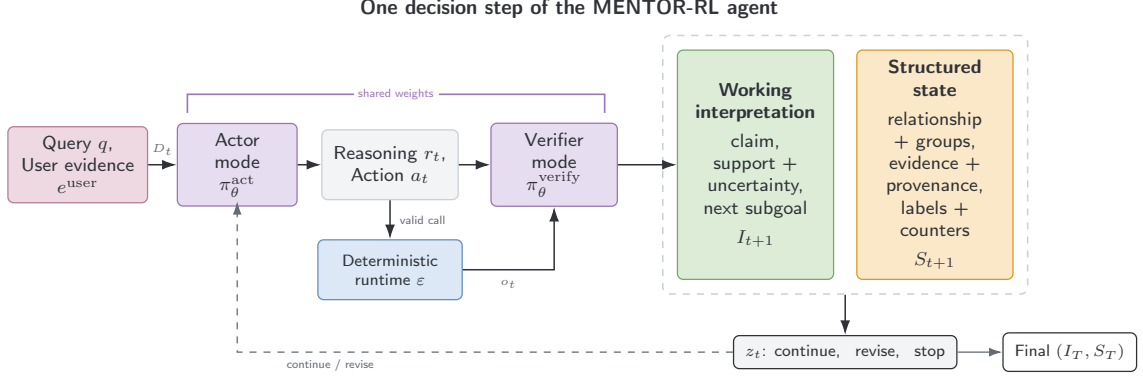


Figure 3: One iteration of the act–tool–verify loop. Actor and verifier modes share model weights, while the graph runtime remains deterministic. Verification produces synchronized human-readable and machine-readable views of the current hypothesis and a decision to continue, revise, or stop.

Agent Loop and Structured State

We model mechanistic exploration as a sequential decision process executed by a single policy with an internal self-verification step. The loop begins when the user provides a query q and optional evidence e , which may include free text, seed genes, or a multiplex. These initialize the working interpretation I_0 and the structured state S_0 .

At each step the policy first acts. Given the current decision context, it emits a textual reasoning step r_t naming its current subgoal, together with an optional tool call a_t . A tool call is executed by the deterministic runtime ε of Section 0.2.4 to produce an observation o_t ; when no call is made, the observation is empty. The same policy then verifies. Conditioned on the decision context and on its own reasoning, action, and observation, it updates the interpretation and state and emits a continuation decision z_t of **continue**, **revise**, or **stop**. A decision of **revise** means the next iteration corrects the interpretation and state rather than extending them. One iteration is therefore

$$r_t, a_t \sim \pi_{\theta}^{\text{act}}(\cdot \mid D_t), \quad o_t = \varepsilon(a_t), \quad I_{t+1}, S_{t+1}, z_t \sim \pi_{\theta}^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t),$$

where $D_t = (q, e, I_t, S_t)$ is the decision context and $\varepsilon(\emptyset) = \emptyset$ by convention. The actor and verifier are two prompting modes of the same weights, so biological vocabulary and tool-use competence learned in one mode transfer to the other. We treat the risk that the verifier collapses into agreement with the actor as a monitoring concern during training. The loop repeats until the policy emits **stop** or the decision budget $T_{\max}$ is exhausted, and returns the pair (I_T, S_T) : a human-readable answer grounded in the accumulated evidence, and the machine-readable record used for scoring and provenance.

The working interpretation holds the current mechanistic claim, the main evidence supporting that claim, any uncertainty, conflict, or alternative explanation, and the next subgoal the model is trying to resolve. The structured state stores the same hypothesis in a machine-readable form suitable for tool execution, reward computation, and provenance tracking. It contains:

1. the user-provided anchors (q, e) ,
2. the current relationship status: unknown, partially observed group, validated group, multiple groups, or insufficient support,
3. the current predicted group or groups,
4. the evidence and provenance collected so far,
5. the current predicted mechanistic labels, each tied to the evidence that supports it,
6. the tool-use counters recording total and invalid calls, and
7. the remaining budget and continuation state, with the termination signal recorded at the end of the trajectory.

To ground the agent’s reasoning in evidence, we enforce three operating rules. First, every claim must be supported by retrieved evidence, and the agent must state its uncertainty explicitly or abstain when support is weak. Second, the agent must use the MENTOR-EV dendrogram modules as an index and confirm its hypotheses on the network, following the multiview structure described in Section 0.2.3. Third, the agent must return both a human-readable interpretation and a structured state for record-keeping. The reward function and training curriculum reinforce these behaviors.

Task Cases and Targets

We define three categories of agent-training tasks that build on the lattice representation learned in Stage 1: module, connectivity, and projection tasks. Each has targets that provide deterministic rewards and support evaluation.

Module tasks fall into four cases: partial-group completion, mechanism explanation, group refinement, and recognition of no shared mechanism. In partial-group completion, the agent receives a module with some genes held out and must recover the complete module. In mechanism explanation, the agent receives a complete module and must identify its supporting mechanistic evidence without adding or removing genes. In group refinement, the agent receives a module with distractor genes and must remove them to recover a mechanistically coherent group. In the no-shared-mechanism case, the agent receives a conflict-free set that is not mechanistically coherent and must report that the evidence provides insufficient support for a shared mechanism.

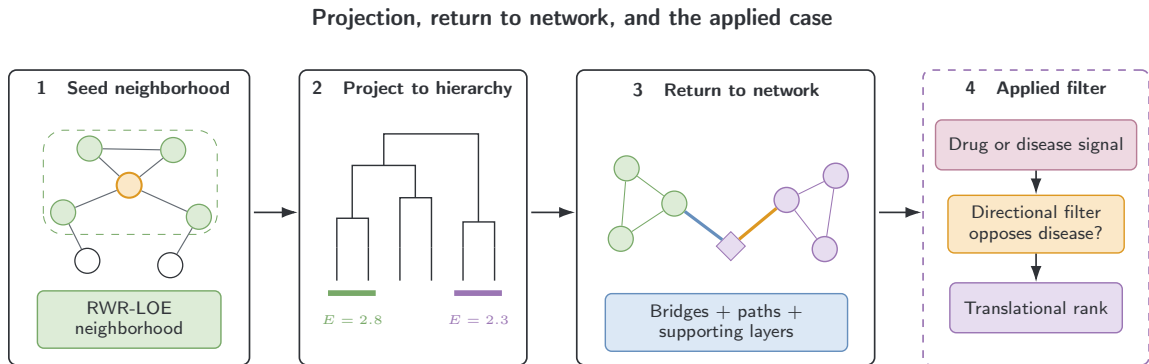


Figure 4: Projection and return to network, with the applied drug-target or disease-signal extension. A seeded RWR-LOE neighborhood is enriched against the hierarchy; significant, potentially distant clades are then connected and interpreted on the multiplex. Directional and translational filters are applied only after this network confirmation.

Connectivity tasks join the two module views through the network that underlies both. The agent is given two clades that are distant in the MENTOR-EV hierarchy and must identify a short path between them, along with the layers that support that path. The agent must also identify any bridge nodes that connect the two modules, and provide mechanistic evidence for the role of those bridge nodes. This task tests the agent’s ability to reason about the underlying multiplex structure and recover connectivity that is not captured by the bottom-up or top-down views alone.

Finally, we train the model on projection tasks, which require the agent to project a seed gene’s RWR-LOE neighborhood onto the MENTOR-EV hierarchy and identify the clades that are enriched for that neighborhood. Because the neighborhood was built by propagation over the network, enrichment landing in several distant clades is evidence of a single connected system rather than unrelated processes, and the agent must read it that way. It must then recover the bridge nodes and short paths that connect those clades and provide mechanistic evidence for their role. Projection tasks are scored on three targets separately: the projection itself, the connectivity recovered to support it, and the interpretation placed on that connectivity. A candidate passes only when all three clear, since a correct projection paired with a failed connectivity recovery is precisely the error that the network view exists to prevent.

The agent curriculum culminates in the applied case, in which the projected seed is a drug target or a disease signal. Here the agent must additionally apply directional and translational filtering. Directional filtering asks whether the candidate intervention opposes the disease-associated direction within a specific module rather than merely acting near it. Translational filtering ranks surviving candidates on evidence strength, adverse-event risk, brain penetration, feasibility, and testability. This case is the repurposing workflow that motivates the disease projections described in Section 0.2.1.

Module tasks use scorer-only membership targets drawn from RWR-LOE neighborhoods and MENTOR-EV modules. Recovery and refinement target the complete module, explanation preserves the supplied module, and **none** has an empty target. Because these computational modules usually lack per-entry experimental annotations, explanations are scored against evidence retrieved during exploration rather than a hidden biological label.

The runtime supplies family-specific targets for connectivity and projection: optimal paths, connector and bridge nodes, and supporting layers for connectivity; and per-clade enrichment, cross-clade connectivity, and allowed or disallowed interpretations for projection. Any optimal path is accepted. All targets remain hidden from the agent, and evidence grounding is scored consistently across task families.

Task Corpora

We generate module tasks from top-down MENTOR-EV modules and bottom-up RWR-LOE neighborhoods sampled across the lattice. For each module of size n , we deterministically materialize up to four instances: **explanation** uses the full module, **recovery** drops k_{drop} genes, **refinement** adds k_{add} distractors, and **none** constructs an unrelated pseudo-module with an empty target. Perturbation size increases with task difficulty and module size.

The two module sources differ only in where distractors come from. For MENTOR-EV modules, noise and **none** genes are drawn from difficulty-modulated *dendrogram distance bands*, defined by walking up the module’s ancestry in the tree, so that difficulty controls how topologically close the distractors are to the module. For RWR-LOE modules we substitute rank bands, drawing from ranks just beyond, moderately beyond, or far beyond the seed’s elbow cutoff. In both cases **none** genes are additionally constrained to be conflict-free, so that the sampled genes do not form a coherent module of their own, and all selection is deterministic given a fixed seed, making the corpus reproducible. Module-size bins are assigned after construction from the realized module sizes, and all splits follow the protocol of Section 0.2.4.

Connectivity instances pair sampled clades at specified tree distances; projection instances sample seed genes. The runtime then materializes the path, layer-support, enrichment, and connectivity targets defined in Section 0.2.5.

Trajectory Generation

Trajectories are generated by shared-prefix branch exploration. At each decision point the policy proposes several candidate continuations from the same state, the runtime executes any valid tool calls among them, and the resulting branches are scored against one another. One branch is retained as the trajectory, while eligible alternatives become material for preference pairs. The procedure is described below.

Given a sampled task and initialized state, we sample multiple action candidates $(r_{t,i}, a_{t,i})$ from the current decision context D_t and execute valid tool calls in the

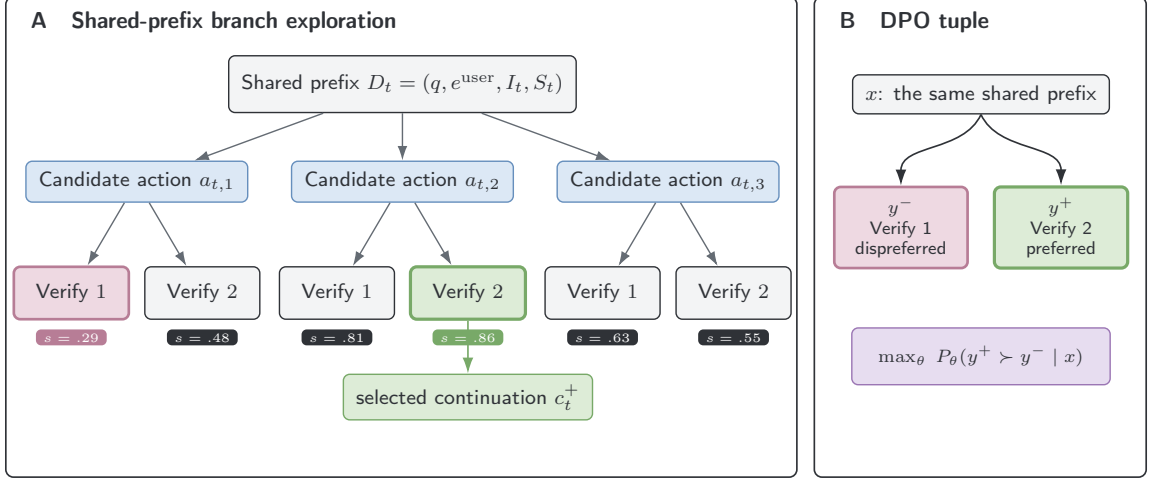


Figure 5: Shared-prefix branch exploration and DPO preference construction. Candidate actions and verifier updates are compared only after the same decision context. Task-quality selection chooses the retained continuation; eligible lower-scoring branches from that pool become dispreferred continuations when the normalized margin exceeds τ .

deterministic graph runtime. To ensure that candidates with the same prefix explore distinct regions of the action space, we use verbalized sampling based on the visible task framing. For example, a query asking the agent to recover missing members biases candidates toward random-walk expansion, neighborhood expansion, subgraph validation, or a direct decision, while a query asking whether a shared mechanism is supported directs the model toward disconfirming graph evidence. When a sampling round for recovery or refinement fails to produce valid tool calls, we issue a tool-coverage retry that requires a valid graph operator whenever one can be formed, preventing the branch pool from degenerating into tool-free continuations. Conditioned on each candidate action and observation, the same policy samples multiple verification candidates, yielding a tree of candidate next states rooted at a shared prefix. Figure 5 visualizes this branching scheme.

Each candidate branch is assigned a deterministic local score computed from the structured state, runtime outputs, and hidden scorer-only targets for its task family. These local scores are used to rank same-prefix candidates and mine DPO preference pairs against that selection. Because several branches frequently tie or fall within a small margin of each other, continuation selection does not reduce to a strict $\arg \max$, but instead, within a tolerance margin ϵ of the best normalized score, ties are broken with task-specific heuristics such as prompted seed gene retention and tool-supported evidence, pushing the logged trajectory to favor well-grounded candidates over those that score highly by chance. We defer the formal score definitions to Section 0.2.5.

Model prompts carry a compact state rather than a growing transcript. Each prompt supplies the current working interpretation, the relationship status, the

predicted groups, the mechanistic labels tied to the evidence handles that support them, the tool and evidence counters, and the current observation. Prior observations are not appended, so prompt length stays bounded as trajectories grow. The full evidence payloads are retained in the logged trajectory artifacts, where they remain available for scoring, audit, preference-pair mining, and expert review.

We provide several query modes and evidence types to represent the diversity of real-world inputs. During training, we sample queries from task-specific templates paraphrased by an open-weight model to ensure linguistic diversity. The agent receives either minimal evidence, consisting only of the query and associated seed genes, or graph-supported evidence, which includes a relevant multiplex subgraph. At inference time, the agent receives a natural-language query with optional graph or seed evidence. During trajectory generation, the requested operation is conveyed through this natural-language framing, but the agent never sees the canonical task-type label, true relationship status, or hidden target. Invalid tool calls return an empty observation and count toward the efficiency penalty, encouraging concise and effective reasoning.

We define a three-stage curriculum for the mechanistic agent, which gradually increases the complexity of tasks and length of trajectories. The first stage focuses on module tasks, where the agent learns to recover and refine modules from partial or noisy inputs. The second stage introduces connectivity tasks, where the agent must identify paths and bridge nodes between distant clades. The final stage focuses on projection tasks, where the agent must project neighborhoods onto the hierarchy and recover mechanistic evidence for the resulting connections, synthesizing the skills learned in the previous stages. Each stage builds on the previous one, ensuring that the agent develops a comprehensive understanding of the multiplex structure and the mechanistic relationships within it.

Reward Design

We define training-time scores over structured state transitions, deterministic runtime metadata, and hidden scorer-only targets specific to each task family; natural-language reasoning is not scored directly. This keeps scoring reproducible, machine-parseable, and scalable without relying on human judges during training. Every score has four shared components: schema compliance, task correctness, evidence-grounded mechanistic quality, and efficiency.

Because DPO compares continuations after the same prefix, we score a candidate branch b by its change in task correctness and mechanistic quality:

$$s_{t,b}^{\text{local}} = w_s s_{t,b}^{\text{schema}} + w_q \Delta q_{t,b}^{\text{task}} + w_m \Delta m_{t,b} - w_e p_{t,b}^{\text{call}} - p_{t,b}^{\text{guard}}. \quad (1)$$

Here, $q^{\text{task}}(S) \in [0, 1]$ is the correctness score associated with the sampled task family, $\Delta q_{t,b}^{\text{task}} = q^{\text{task}}(S_{t+1,b}) - q^{\text{task}}(S_t)$, and $\Delta m_{t,b} = m(S_{t+1,b}) - m(S_t)$. The schema term is one only when the candidate state and any tool request satisfy their typed validators. The guard penalty covers task-incompatible behavior; for example, an

explanation branch is penalized for removing members from an already-complete module, and mechanistic improvement is clamped to be non-positive whenever a branch degrades task correctness.

For recovery, refinement, and explanation tasks, task correctness is the best match between any predicted group and the hidden module target C :

$$q^{\text{module}}(S; C, C_{\text{type}}) = \max_{G \in \mathcal{G}(S)} [\alpha J(G, C) + \beta P(G, C) + \gamma R(G, C)], \quad (2)$$

where $\mathcal{G}(S)$ is the set of predicted groups in structured state S , with an empty group used when no prediction is present. Recovery upweights recall ($\alpha, \beta, \gamma = 0.2, 0.2, 0.6$), refinement upweights precision ($0.2, 0.6, 0.2$), and explanation weights Jaccard, precision, and recall equally. This formulation supports a multiple-groups state while retaining a single-module hidden target. The local module improvement is the task-generic delta

$$\Delta q_{t,b}^{\text{module}} = q^{\text{module}}(S_{t+1,b}) - q^{\text{module}}(S_t). \quad (3)$$

For a **none** task, the correctness score instead rewards calibrated abstention:

$$q^{\text{none}}(S) = 0.6 \mathbf{1}[rel(S) = \text{insufficient_support}] + 0.4 \mathbf{1}[\mathcal{G}(S) = \emptyset]. \quad (4)$$

Thus, **none** is an instantiation of the shared task-correctness term rather than a separate reward pathway. The multiple-groups status remains a supported state, but it receives no separate score unless the corpus later materializes a corresponding task family.

For connectivity tasks, q^{conn} combines path validity and optimality, connector and bridge-node recovery, and supporting-layer recovery. If v_{path} denotes the fraction of returned paths that are valid and optimally short, B the returned connector and bridge nodes, and L the returned supporting layers, then

$$q^{\text{conn}}(S) = \eta_p v_{\text{path}}(S) + \eta_b J(B(S), B^*) + \eta_l J(L(S), L^*), \quad \eta_p + \eta_b + \eta_l = 1. \quad (5)$$

Any optimal path is accepted rather than requiring a single reference path. For projection tasks, the correctness score preserves the operation’s three distinct targets:

$$q^{\text{proj}}(S) = \eta_e q^{\text{enrich}}(S) + \eta_c q^{\text{conn}}(S) + \eta_i q^{\text{interp}}(S), \quad \eta_e + \eta_c + \eta_i = 1. \quad (6)$$

The enrichment component scores significant-clade recovery and copied runtime values, the connectivity component scores the return to the network, and the interpretation component scores the structured allowed and disallowed readings. These components provide dense local feedback, but a completed projection passes only when all three clear their individual thresholds, so projection accuracy cannot

compensate for failed connectivity recovery. Directional and translational fields in the applied case are scored within the structured interpretation component when present.

We define mechanistic quality $m(S) \in [0, 1]$ from evidence the agent has retrieved. Let g , d , e , a , n , and h denote grounding, specificity, enrichment support, annotation support, network support, and stability across query subsets, respectively. We define consensus $c = \max(e, a)$, cross-tool agreement $x = \min(e, a)$, annotation-evidence strength $A = \max(e, 0.75a)$, total evidence strength $E = \max(A, 0.50n)$, and network agreement $\bar{n} = n$ when $c > 0$ and $0.35n$ otherwise. For a state that asserts a mechanism,

$$m^{\text{claim}} = \kappa (0.25g + 0.25c + 0.18d + 0.12\bar{n} + 0.10h + 0.10x), \quad (7)$$

where $\kappa = 0.35$ when a claim is present but $E \leq 0.05$, and $\kappa = 1$ otherwise. For an abstaining state, $m^{\text{abs}} = 0.60u + 0.20\bar{n} + 0.20g$, where u rewards withholding a claim as evidence strength decreases. On a non-**none** task, this abstention score is forced to zero when $A \leq 0.05$, preventing unsupported abstention from earning mechanistic reward when a target is recoverable. Grounding requires claimed entities or labels to occur in observed evidence, while specificity penalizes generic claims.

The local efficiency term penalizes only behavior that differs among candidates after the same prefix. If $n_{\text{call}}^{t,b}$ and $n_{\text{inv}}^{t,b}$ are the tool calls and invalid calls contributed by branch b , then

$$p_{t,b}^{\text{call}} = \lambda_{\text{inv}} \frac{n_{\text{inv}}^{t,b}}{\max(1, n_{\text{call}}^{t,b})}. \quad (8)$$

Schema-invalid requests, exact duplicates, missing observations, and error observations count as invalid; a branch that makes no tool call receives no call penalty. Trajectory length is omitted locally because it is constant across same-prefix candidates and therefore cannot affect their DPO ranking.

Terminal Reward Decomposition for GRPO

Unlike DPO, which compares local same-prefix branches, policy gradient methods require a scalar reward for a completed rollout. We therefore define a terminal reward over the final structured state and trajectory log:

$$\begin{aligned} R_T = & w_s^T s_{\text{schema}}^T + w_q^T q_T^{\text{task}} + w_{\Delta q}^T (q_T^{\text{task}} - q_0^{\text{task}}) \\ & + w_m^T m_T + w_{\Delta m}^T (m_T - m_0) - w_e^T p_{\text{eff}}^T. \end{aligned} \quad (9)$$

Here, q_T^{task} uses the same family-specific definition as local scoring, so absolute and cumulative correctness apply equally to module, **none**, connectivity, and projection tasks. The terminal mechanistic terms use the final evidence-grounded score and its change from initialization. Mechanistic credit is capped when the task-specific correctness gate fails, preventing a fluent interpretation from compensating for an incorrect module, path, or projection. The terminal schema score additionally

requires a valid final state and a recorded termination mode, whether the agent stops voluntarily or exhausts its budget.

The terminal efficiency penalty aggregates trajectory length and invalid tool use across the rollout:

$$p_{\text{eff}}^T = \lambda_1 \frac{T}{T_{\text{max}}} + \lambda_2 \frac{\sum_{t=0}^{T-1} n_{\text{inv}}^t}{\max\left(1, \sum_{t=0}^{T-1} n_{\text{call}}^t\right)}. \quad (10)$$

The denominator convention makes the invalid-call term zero for a trajectory that makes no tool calls. Unlike the local score, the terminal reward includes T/T_{max} because complete rollouts can differ in length.

At each decision point we form DPO preference tuples from the candidate pool, pairing the shared-prefix context with one preferred and one dispreferred continuation. Each continuation carries its reasoning step, tool action, verifier update, and continuation decision. Scores are min-max normalized within each pool first, so that the selection margin has the same meaning across task types and trajectory lengths.

The preferred candidate is the branch selected as the trajectory continuation rather than the strict score maximizer, since selection resolves near-ties using task-specific heuristics and therefore reflects the move judged best for the task. Candidates within a margin τ of the selected branch are discarded so that no pair is built from a near tie, and the remainder form the dispreferred pool. From that pool we draw examples across `easy`, `medium`, and `hard` difficulty bins, taking the lowest, median, and highest scoring remaining candidates respectively. Each pair is stored with its bin, task type, decision step, scores, and margin, and the corpus is rebalanced across task types and bins so that training is not dominated by the categories that happen to yield the most informative pairs.

Each retained trajectory is also logged as a sequence of turn records, from which we render short per-step findings and a final mechanistic summary. These become supervised examples and preference pairs for the training pipeline described in Section 0.2.6. The local scores above supply the preference signal for DPO, while the terminal reward supplies the scalar return optimized during GRPO.

0.2.6 Training Pipeline and Curriculum

We first train the policy with SFT using the curriculum in Section 0.2.4. Training will run on OLCF’s Frontier supercomputer [3], with Slurm [52] for job scheduling, Hugging Face and TRL [50] for optimization, DeepSpeed ZeRO-3 [37] for model parallelism, Low-Rank Adaptation for parameter-efficient fine-tuning [18], and Weights & Biases [8] for convergence tracking. We use gpt-oss-120b [32] as the base policy and early-stopping checkpoints to prevent overtraining.

After SFT, the trajectory method in Section 0.2.5 generates preference pairs scored by Section 0.2.5. The selected continuation is c_t^+ , and eligible alternatives across the `easy`, `medium`, and `hard` bins supply c_t^- . DPO increases the likelihood of c_t^+ relative

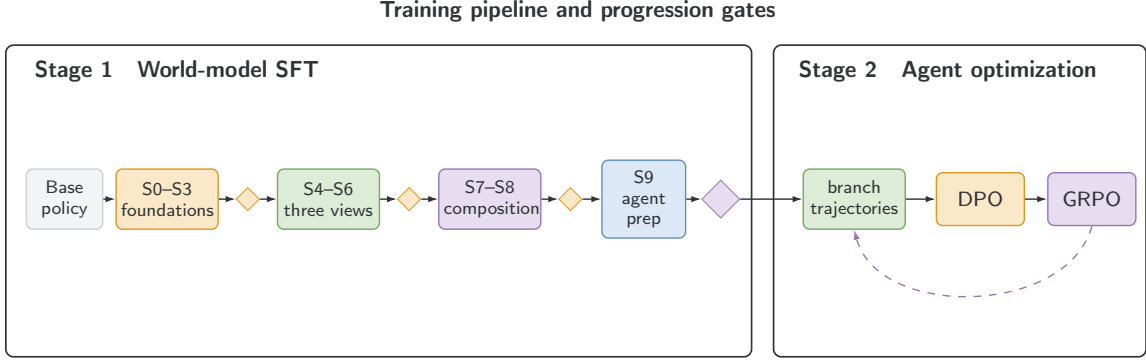


Figure 6: The gated two-stage training pipeline. Stage 1 advances through supervised curriculum steps only after held-out gates clear. Its checkpoint initializes trajectory generation and preference optimization in Stage 2, followed by online GRPO. Checkpoint evidence accumulates from structural evaluations through agent behavior and expert review.

to c_t^- under a frozen reference policy without a separate reward model [36]. We regenerate trajectories from updated checkpoints as training progresses.

The DPO checkpoint then initializes GRPO for long-horizon exploration. Each rollout receives an advantage equal to its terminal reward standardized within its rollout group, removing the need for annotated trajectories or a learned value model [41]. The DPO checkpoint remains the fixed reference policy, anchoring online exploration while GRPO promotes trajectories beyond the logged preference data.

We monitor tool-use entropy, trajectory diversity, and the single-pass RWR shortcut, in which one expansion recovers much of a module without meaningful exploration. If shortcuts emerge, we introduce adversarial modules or a diversity bonus; these monitors must clear before advancement.

Each transition is gated. Stage 1 uses the per-step gates in Table 2; DPO must improve module recovery and mechanistic grounding over SFT on held-out splits; and GRPO must improve the flagship evaluations with shortcut monitors clear. Deployment additionally requires expert preference to rise across the base, SFT, DPO, and GRPO checkpoints under Section 0.2.7.

Difficulty scales throughout optimization. Stage 1 independently advances book mode, operator complexity, lattice tier, cut-height granularity, and distractor band. DPO moves from **easy** to **hard** pairs and from module recovery and refinement to connectivity and projection. GRPO increases the horizon $T_{\max}$, realized module size, perturbation size, distractor proximity, and lattice tier up to higher-order brain combinations. Batches remain balanced across task types, difficulty bins, and both module sources.

0.2.7 Evaluation

Stage 1 Evaluation: World Model SFT

Stage 1 advances through the held-out, step-specific gates in Table 2. Two flagship evaluations then measure overall mechanistic reasoning: held-out context hierarchy generation, and projection and return to network. Probing and intervention provide a cross-cutting causal gate.

For held-out context hierarchy generation, the model receives an unseen context’s active-layer descriptor and a query gene set, then returns pairwise co-membership, lowest-common-ancestor depth, and a partition. We score these against the runtime dendrogram. Because hierarchies diverge across contexts, success requires context-specific structure rather than a memorized hierarchy.

Projection and return to network follows Section 0.2.5: a target gene seeds an RWR-LOE neighborhood, the neighborhood is enriched across the hierarchy, and the model recovers the paths and bridge nodes connecting significant clades. This tests whether the model can unify the multiplex views into one mechanistic interpretation.

To test whether lattice operators are internalized, multilayer-perceptron probes recover edge presence, layer membership, clade co-membership, and connector status from model activations. We then intervene on those activations and measure whether answers change, distinguishing causal structure from mere decodability. This is a gate rather than a diagnostic and is repeated under the lean-context condition without the full rank-vector legend. Table 3 summarizes both flagships and the probe gate.

Stage 2 Evaluation: Mechanistic Agent DPO and GRPO

We calculate evaluation metrics by parsing the model’s terminal structured state S_T , enabling scalable checkpoint evaluation. All evaluations use a held-out set of MENTOR-EV- and RWR-LOE-derived modules from the brain multiplex. We measure module and cross-clade path recovery with Jaccard, precision, recall, and F1. Jaccard penalizes both missing and extra genes, precision and recall separate those error directions, and F1 provides a balanced summary. We also report mechanistic-grounding quality to test whether claims align with the evidence retrieved by the model.

We report the weighted composite reward R_T , defined in Section 0.2.5, to compare reward magnitude on validation and holdout splits. For the **none** task type, where $C = \emptyset$, we parse S_T for the predicted relationship rel_T . In addition to global accuracy, we distinguish the two error directions: claiming that no mechanism exists for a related gene set and fabricating a mechanism for unrelated entities. We also report mean trajectory length, budget utilization, and invalid-tool-call rate, enabling comparison of holdout and validation efficiency under Equation 10.

We stratify module-task results by module size, noise, task type, and module source. We also stratify by lattice tier and cut-height granularity to measure performance across regions of the lattice. Each axis is divided into three discrete

Table 3: Scoring for the two Stage 1 flagships and the cross-cutting probe gate. Components are reported separately; projection passes only when projection, connectivity recovery, and interpretation all clear their thresholds.

Evaluation or gate	Component	Metric
Held-out context hierarchy generation	Partition agreement	Adjusted Rand index of the predicted partition against the runtime dendrogram of the held-out context
	Clade co-membership	Same-clade accuracy over query gene pairs
	Ancestry	Lowest common ancestor depth accuracy per gene pair
Projection and return to network	Projection	Jaccard of the predicted significant clades against the runtime enrichment, with per-clade score and q -value copy accuracy
	Connectivity recovery	Jaccard of recovered connector and bridge nodes, path validity rate, and the fraction of recovered paths at optimal length
	Interpretation	Accuracy of the connected-system reading, and the rate at which the disallowed reading that tree distance implies separation is asserted
Probing and intervention gate	Decodability	Probe classification accuracy for edge presence, layer membership, clade co-membership, and connector status
	Causality	Change in the model’s answer when the probed activation is modulated to flip its predicted value
	Scaffold independence	Probe and interpretation accuracy retained under the lean context condition, without the full rank-vector legend

bins. These metrics show how MENTOR-RL performs as difficulty increases and task type changes; comparison by module source reveals whether the model generalizes across construction methods or favors one module type.

Alongside empirical validation, we conduct blinded comparative evaluations of domain experts’ model preferences. First, an inter-model preference test compares interpretation outputs from MENTOR-RL, GPT-5 [43], gpt-oss-120b [32], LLaMA 4 [2], Kimi K2 [46], and Nemotron 3 Super [31]. Between 5 and 15 domain experts receive paired samples containing a MENTOR-RL output and an output from another randomly selected model. We report the proportion of pairs in which experts prefer MENTOR-RL. A subset of pairs is shared across raters to measure inter-expert reliability.

We then use the same protocol for intra-model preference testing across the base, SFT, DPO, and GRPO-trained MENTOR-RL checkpoints. These blinded comparisons assess characteristics such as style, tone, and concision that empirical metrics do not capture easily.

Ablation Studies

To ensure that each component of our methodology contributes to an improvement in mechanistic interpretation, we will ablate both training stages and design elements. All ablations will be evaluated using the holdout metrics defined in Section 0.2.7.

We ablate phases of the training pipeline to test whether each stage improves on the preceding checkpoint. First, we compare the SFT-only checkpoint with the SFT-and-DPO checkpoint to test whether preference optimization improves interpretation. Next, we compare the DPO checkpoint with the full pipeline to determine whether long-horizon RL improves on local preference training. Finally, we ablate Stage 1 entirely, training the agent from the base policy without the world-model curriculum and comparing it with the complete pipeline on module-recovery, connectivity, and projection tasks. This ablation tests the proposal’s central claim that the world model is necessary rather than merely helpful. Preliminary trajectory diagnostics in Section 0.4.1 indicate what to expect: an untrained policy prunes distractors successfully but fails to recover held-out members or ground its claims in retrieved evidence.

We also ablate architectural and reward decisions. First, we remove the policy’s verifier mode to test whether self-verification improves results. Second, we remove the efficiency penalty p^{eff} to test whether MENTOR-RL learns reasonable trajectory lengths without regularization. Finally, we remove progressive difficulty bins and train on all bins simultaneously to test whether the model can learn from hard examples without curriculum staging.

Finally, we ablate two Stage 1 presentation choices. For S4 rank vectors, we compare a structured table, an ordinal sequence with the elbow marked, and a hybrid using bands and calibration anchors; we fix the best rendering before the full build. We also vary the context-fade schedule, progressively removing legends, anchors, and linked hierarchy views to test whether accuracy survives withdrawal of the scaffold.

0.3 Expected Results

0.3.1 Expected Empirical Results

We expect MENTOR-RL to recover modules and cross-module paths accurately in held-out lattice contexts, measured by Jaccard, precision, recall, and F1; project targets through RWR-LOE neighborhoods to enriched clades and connecting paths; and ground interpretations in retrieved evidence. Probes should decode lattice structure, and interventions should show that it is causal to reasoning. Improvement

across SFT, DPO, and GRPO checkpoints on unseen contexts would indicate out-of-distribution generalization.

0.3.2 Expected Behavioral Results

We expect the act–verify loop to reduce invalid calls and unsupported claims, the efficiency penalty to shorten trajectories without reducing recovery, and expert preference to rise across the base, SFT, DPO, and GRPO checkpoints. The corresponding verifier, efficiency, and checkpoint ablations in Section 0.2.7 test each claim.

0.3.3 Deliverables

We will release, under licenses compatible with the underlying data sources, the following artifacts:

1. the lattice construction code and context ontology,
2. the lattice-grounded training environment, including the `ctypes` multiplex runtime, tool vocabulary, and deterministic scoring functions,
3. train, validation, and test split indices at the context, subtree, and tuple level, a leakage audit, and the code required to reconstruct the raw training corpus and cached API responses,
4. trained MENTOR-RL checkpoints for each stage of the pipeline (SFT stages S0-S9, DPO, and GRPO),
5. the flagship harnesses, including held-out contexts and reserved target genes, and
6. training and generation code, curriculum configurations, and logged trajectories to support reproducible re-training and ablation.

By releasing a deterministically-scored, open-source mechanistic exploration world model and agent together with its training environment, we aim to lower the cost of biological interpretation research and establish a reproducible benchmark that does not require proprietary model access. The scoring framework is domain-specific, but the training process to learn operators over networks in a lattice structure is transferable to other scientific settings with curated ground truth.

0.4 Discussion

0.4.1 Preliminary Results

Several preliminary results support the feasibility of the proposed approach. First, we demonstrated full fine-tuning of the 120-billion-parameter open-weight gpt-oss-120b model [32] using data, tensor, and expert parallelism on OLCF’s Frontier supercomputer. We have trained on up to 128 nodes. This infrastructure, together with the ability to test smaller models such as gpt-oss-20b under the same training and evaluation pipeline, meets the capacity requirements for the multiplex-lattice dataset.

We also implemented the Stage 2 trajectory-generation loop (Section 0.2.5) using the same gpt-oss-120b model. The loop runs end-to-end, provides a dashboard for run inspection, and produced no parsing errors in the preliminary runs. However, initializing trajectory generation from a base model without Stage 1 training yields poor performance: the protocol succeeds on refinement tasks but fails on recovery tasks and provides weak mechanistic grounding. These results support the need for Stage 1 to provide domain knowledge and tool-calling capabilities before Stage 2. Because they are based on a small number of runs and do not include the full evaluation suite, we will evaluate the loop more thoroughly after Stage 1 training.

Finally, we generated the corpora for the Stage 1 S0 and S1 curriculum steps. Our current focus is S0 training, which covers Ensembl-to-symbol and symbol-to-Ensembl mapping, as well as ambiguous-symbol resolution.

To begin Stage 1 training, we tested four tokenization approaches for mapping tasks. First, we used the gpt-oss-120b base tokenizer, fitted to its pretraining corpus with byte-pair encoding (BPE), as a control. Second, we fit a BPE tokenizer only to Ensembl identifiers and gene symbols, then concatenated the resulting vocabulary with the base vocabulary; we call this **domain-BPE** tokenization. Third, we encoded the Ensembl prefix (ENSG) as an atomic token, fit BPE to the remaining identifier suffixes and gene symbols, and concatenated that vocabulary with the base vocabulary; we call this **atomic+domain-BPE** tokenization. Finally, we represented each Ensembl identifier using atomic tokens for its prefix and accession body, represented each gene symbol with one atomic token, and tokenized module identifiers compositionally; we call this **atomic** tokenization.

Our preliminary results show that base tokenization performs poorly on mapping tasks. We trained gpt-oss-120b on approximately 2,000 questions from a small S0 subset under each tokenization approach and evaluated it on 1,000 hard-subset questions. Overall accuracy increased from 45.6% with the base tokenizer to 85.9% with atomic tokenization. The mapping-specific gains were larger: symbol-to-Ensembl accuracy increased from 8.3% to 100%, and Ensembl-to-symbol accuracy increased from 5.6% to 100%. These results select atomic tokenization for S0, pending confirmation at full vocabulary scale with held-out and low-frequency entities.

0.4.2 Plan of Work

We will complete S0–S9 sequentially, materializing each corpus after the preceding gate clears; S0 is underway. The resulting Stage 1 checkpoint will undergo the two flagship evaluations, the cross-cutting probe gate, and curriculum ablations before generating Stage 2 trajectories. DPO on those trajectories will initialize GRPO, followed by the final evaluations, ablations, and deliverables in Section [0.3.3](#).

0.4.3 Limitations and Future Work

We identify several limitations and possible extensions. First, the lattice structure depends on the assumption that different contexts generate meaningfully different hierarchies. We will validate this assumption by comparing hierarchies across contexts; insufficient divergence would limit the lattice’s utility. The approach also requires substantial compute to construct the RWR-LOE and MENTOR-EV corpora and train the model on the multiplex-lattice dataset. Although we have access to OLCF’s Frontier supercomputer, these requirements may limit accessibility for other researchers.

Future work can extend the lattice to other organ systems and disease states, yielding a more comprehensive model checkpoint. Extending the lattice across species could support cross-species mechanistic reasoning. Additional evidence modes and curriculum stages could also teach more operators and support more complex reasoning across modalities.

Bibliography

- [1] Shibli Abdulla et al. “CZ CELLxGENE Discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data”. In: *Nucleic Acids Research* 53.D1 (2025), pp. D886–D900.
- [2] Redacted by arXiv. *The Llama 4 Herd: Architecture, Training, Evaluation, and Deployment Notes*. 2026. arXiv: [2601.11659 \[cs.SE\]](https://arxiv.org/abs/2601.11659). URL: <https://arxiv.org/abs/2601.11659>.
- [3] Scott Atchley et al. “Frontier: Exploring Exascale”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC ’23. Denver, CO, USA: Association for Computing Machinery, 2023. ISBN: 9798400701092. DOI: [10.1145/3581784.3607089](https://doi.org/10.1145/3581784.3607089). URL: <https://doi.org/10.1145/3581784.3607089>.
- [4] Albert-László Barabási, Natali Gulbahce, and Joseph Loscalzo. “Network medicine: a network-based approach to human disease”. In: *Nature reviews genetics* 12.1 (2011), pp. 56–68.
- [5] Albert-Laszlo Barabasi and Zoltan N Oltvai. “Network biology: understanding the cell’s functional organization”. In: *Nature reviews genetics* 5.2 (2004), pp. 101–113.
- [6] Sumanta Basu et al. “Iterative random forests to discover predictive and stable high-order interactions”. In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948.
- [7] Bgee Group. *HsapDv: Human Developmental Stages Ontology*. OBO Foundry. <http://purl.obolibrary.org/obo/hsapdv.owl>. 2024.
- [8] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: <https://www.wandb.com/>.
- [9] Katy Börner et al. “Anatomical structures, cell types and biomarkers of the Human Reference Atlas”. In: *Nature Cell Biology* 23.11 (2021), pp. 1117–1128.
- [10] Yiqun Chen and James Zou. “Simple and effective embedding model for single-cell biology built from ChatGPT”. In: *Nature biomedical engineering* 9.4 (2025), pp. 483–493.
- [11] Paul F Christiano et al. “Deep reinforcement learning from human preferences”. In: *Advances in neural information processing systems* 30 (2017).

- [12] Ashley Cliff et al. “A high-performance computing implementation of iterative random forest for the creation of predictive expression networks”. In: *Genes* 10.12 (2019), p. 996.
- [13] Lenore Cowen et al. “Network propagation: a universal amplifier of genetic associations”. In: *Nature Reviews Genetics* 18.9 (2017), pp. 551–562.
- [14] Manlio De Domenico et al. “Mathematical formulation of multilayer networks”. In: *Physical Review X* 3.4 (2013), p. 041022.
- [15] Alexander D Diehl et al. “The Cell Ontology 2016: enhanced content, modularization, and ontology interoperability”. In: *Journal of Biomedical Semantics* 7 (2016), p. 44.
- [16] Daya Guo et al. “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning”. In: *arXiv preprint arXiv:2501.12948* (2025).
- [17] Leland H Hartwell et al. “From molecular to modular cell biology”. In: *Nature* 402.Suppl 6761 (1999), pp. C47–C52.
- [18] Edward J Hu et al. “Lora: Low-rank adaptation of large language models.” In: *Iclr* 1.2 (2022), p. 3.
- [19] ISO. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. Geneva, Switzerland: International Organization for Standardization, Feb. 2012. URL: <http://www.iso.org>.
- [20] David Kainer et al. “RWRtoolkit: multi-omic network analysis using random walks on multiplex networks in any species”. In: *GigaScience* 14 (2025), gfa028.
- [21] Chan Yeong Kim et al. “HumanNet v3: an improved database of human gene networks for disease research”. In: *Nucleic Acids Research* 50.D1 (2022), pp. D632–D639.
- [22] Sebastian Köhler et al. “Walking the interactome for prioritization of candidate disease genes”. In: *The American Journal of Human Genetics* 82.4 (2008), pp. 949–958.
- [23] Takeshi Kojima et al. “Large language models are zero-shot reasoners”. In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.
- [24] Kenneth Li et al. “Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [25] Pan Lu et al. “Eubiota: Modular Agentic AI for Autonomous Discovery in the Gut Microbiome”. In: *bioRxiv* (2026). DOI: [10.64898/2026.02.27.708412](https://doi.org/10.64898/2026.02.27.708412). eprint: <https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.full.pdf>. URL: <https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412>.
- [26] Aman Madaan et al. “Self-refine: Iterative refinement with self-feedback”. In: *Advances in neural information processing systems* 36 (2023), pp. 46534–46594.

- [27] Jörg Menche et al. “Uncovering disease-disease relationships through the incomplete interactome”. In: *Science* 347.6224 (2015), p. 1257601.
- [28] Peter J Mucha et al. “Community structure in time-dependent, multiscale, and multiplex networks”. In: *science* 328.5980 (2010), pp. 876–878.
- [29] Christopher J Mungall et al. “Uberon, an integrative multi-species anatomy ontology”. In: *Genome Biology* 13.1 (2012), R5.
- [30] Reiichiro Nakano et al. “Webgpt: Browser-assisted question-answering with human feedback”. In: *arXiv preprint arXiv:2112.09332* (2021).
- [31] NVIDIA. *NVIDIA Nemotron 3: Efficient and Open Intelligence*. White Paper. 2025. URL: <https://arxiv.org/abs/2512.20856>.
- [32] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: [2508.10925](https://arxiv.org/abs/2508.10925) [cs.CL]. URL: <https://arxiv.org/abs/2508.10925>.
- [33] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.
- [34] Pranav Putta et al. “Agent q: Advanced reasoning and learning for autonomous ai agents”. In: *arXiv preprint arXiv:2408.07199* (2024).
- [35] Yujia Qin et al. “Toollm: Facilitating large language models to master 16000+ real-world apis”. In: *arXiv preprint arXiv:2307.16789* (2023).
- [36] Rafael Rafailov et al. “Direct preference optimization: Your language model is secretly a reward model”. In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.
- [37] Jeff Rasley et al. “Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters”. In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2020, pp. 3505–3506.
- [38] G. van Rossum. *Python tutorial*. Tech. rep. CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI), May 1995.
- [39] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in neural information processing systems* 36 (2023), pp. 68539–68551.
- [40] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [41] Zhihong Shao et al. “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).

- [42] Noah Shinn et al. “Reflexion: Language agents with verbal reinforcement learning”. In: *Advances in neural information processing systems* 36 (2023), pp. 8634–8652.
- [43] Aaditya Singh et al. *OpenAI GPT-5 System Card*. 2025. arXiv: [2601.03267](https://arxiv.org/abs/2601.03267) [cs.CL]. URL: <https://arxiv.org/abs/2601.03267>.
- [44] Kyle A. Sullivan et al. “MENTOR: Multiplex Embedding of Networks for Team-Based Omics Research”. In: *bioRxiv* (2024). DOI: [10.1101/2024.07.17.603821](https://doi.org/10.1101/2024.07.17.603821).
- [45] Shawn Zheng Kai Tan et al. “Brain Data Standards – A method for building data-driven cell-type ontologies”. In: *Scientific Data* 10 (2023), p. 50.
- [46] Kimi Team et al. *Kimi K2: Open Agentic Intelligence*. 2026. arXiv: [2507.20534](https://arxiv.org/abs/2507.20534) [cs.LG]. URL: <https://arxiv.org/abs/2507.20534>.
- [47] A. H. Vlot et al. “The MENTOR Interpretation Agent: From Network Embeddings to Mechanistic Narratives via Retrieval-Augmented LLMs”. In: *PASC 2025*. 2025.
- [48] Zhizheng Wang et al. “GeneAgent: self-verification language agent for gene-set analysis using domain databases”. In: *Nature Methods* 22.8 (2025), pp. 1677–1685.
- [49] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.
- [50] Leandro von Werra et al. *TRL: Transformers Reinforcement Learning*. <https://github.com/huggingface/trl>. 2020.
- [51] Shunyu Yao et al. “React: Synergizing reasoning and acting in language models”. In: *The eleventh international conference on learning representations*. 2022.
- [52] Andy B. Yoo, Morris A. Jette, and Mark Grondona. “SLURM: Simple Linux Utility for Resource Management”. In: *Job Scheduling Strategies for Parallel Processing*. Springer Berlin Heidelberg, 2003, pp. 44–60. DOI: [10.1007/10968987_3](https://doi.org/10.1007/10968987_3).